

网易AI前端工程师

一面（技术基础）

请简述一下CSS中的 Flex 布局，并说明flex:1是哪些属性的简写？

在处理 AI 聊天回复时，常使用SSE技术，它和WebSocket 有什么区别？

Promise.all 和 Promise.allSettled 的区别是什么？

请手写一个简单的深拷贝函数，需要考虑循环引用。

浏览器中的事件循环（Event Loop）是如何工作的？

TypeScript 中的 interface 和type 有什么区别？

二面（深入原理与项目）

AI聊天界面的流式文本渲染，如果内容非常长，如何保证页面不卡顿？

谈谈 React的Fiber 架构，它解决了什么问题？

在AI 应用中，前端如何计算 Token 数量以防止超过模型上下文限制？

如何实现一个具有“打字机效果”的AI 响应组件？

你们项目中的前端工程化是怎么做的？特别是针对 AI频繁变动的 Prompt。

什么是 RAG（检索增强生成）？前端在 RAG 流程中可以做哪些优化？

手写代码：实现一个带并发限制的 Promise 调度器。

谈谈 Web Worker 的使用场景，在AI 前端应用中它能做什么？

三面（架构与综合能力）

如果让你从零设计一个企业级的 AI Chat桌面端应用，你会如何进行技术选型？

如何设计一个通用的AI 组件库？需要考虑哪些特殊组件？

在 AI落地过程中，前端工程师的价值体现在哪里？

谈谈你对 PromptInjection（提示词注入）的理解，前端能做哪些防护？

当 AI 接口响应非常慢（TTFT 超过5秒）时，你会通过哪些手段优化用户体验？

聊聊你在过去一年中遇到的最难的AI 前端技术问题，你是如何解决的？

你如何看待WebAssembly 在未来 AI 前端领域的发展？

假如项目需要支持“多轮对话的上下文管理”，前端在存储和传输上有什么策略？

一面（技术基础）

1. 请简述一下 CSS 中的 Flex 布局，并说明 flex: 1 是哪些属性的简写？

难度：★

考点：CSS 布局基础

答案要点：

Flex 布局是弹性布局模型，通过设置容器的 display 属性为 flex 来开启，主要用于处理一维空间的元素排列。

- flex: 1 是 flex-grow: 1, flex-shrink: 1, flex-basis: 0% 的简写
- flex-grow 定义放大比例，默认为 0
- flex-shrink 定义缩小比例，默认为 1
- flex-basis 定义在分配多余空间前，项目占据的主轴空间

常见坑：

- 忽略 flex-basis 为 0% 和 auto 的区别
- 不清楚 flex-shrink 在空间不足时的计算逻辑

追问：

- 如果容器宽度 400px，子元素 A 设置 flex: 2，B 设置 flex: 1，它们实际宽度是多少？

2. 在处理 AI 聊天回复时，常使用 SSE 技术，它和 WebSocket 有什么区别？

难度：★★

考点：网络协议、流式传输

答案要点：

SSE (Server-Sent Events) 是一种基于 HTTP 的单向通信协议，允许服务器向客户端推送数据，非常适合 AI 逐字生成的场景。

- 通信方向：SSE 是单向（服务端到客户端），WebSocket 是双向全双工
- 协议基础：SSE 基于标准 HTTP 协议，WebSocket 是独立协议
- 重连机制：SSE 内置自动重连功能，WebSocket 需要手动实现
- 数据格式：SSE 默认传输文本（UTF-8），WebSocket 支持二进制和文本

常见坑：

- 认为 SSE 只能在 HTTP/2 下工作（实际上 HTTP/1.1 也支持，但有连接数限制）
- 忽略 SSE 在浏览器端的最大连接数限制（Chrome 限制为 6 个）

追问：

- 在 HTTP/1.1 下，如何绕过 SSE 的 6 个连接限制？

3. Promise.all 和 Promise.allSettled 的区别是什么？

难度：★

考点：JS 异步编程

答案要点：

这两个方法都用于处理多个并发的 Promise，但对失败状态的处理逻辑完全不同。

- 成功机制：Promise.all 要求全部成功才返回成功，allSettled 会等待所有任务结束
- 失败机制：Promise.all 只要有一个失败就立即 reject，allSettled 不会 reject
- 返回值：allSettled 返回一个包含 status 和 value/reason 的对象数组
- 适用场景：all 适用于强依赖任务，allSettled 适用于互不影响的并行任务

常见坑：

- 在 Promise.all 中没有对单个 promise 进行 catch，导致整个链路中断

追问：

- 如果想实现一个有重试机制的 Promise.all，你会怎么写？

4. 请手写一个简单的深拷贝函数，需要考虑循环引用。

难度：★★

考点：JS 基础、递归、Map

答案要点：

深拷贝需要递归遍历对象属性，并使用 WeakMap 记录已访问的对象以解决循环引用导致的死循环。

- 使用 WeakMap 存储已拷贝的对象
- 区分基本类型和引用类型
- 递归处理对象和数组
- 考虑 Date、RegExp 等特殊对象的处理

代码示例：

代码块

```
1  function deepClone(obj, map = new WeakMap()) {
2    if (obj === null || typeof obj !== 'object') return obj;
3    if (map.has(obj)) return map.get(obj);
4    let target = Array.isArray(obj) ? [] : {};
5    map.set(obj, target);
6    for (let key in obj) {
7      if (obj.hasOwnProperty(key)) {
8        target[key] = deepClone(obj[key], map);
9      }
10   }
11   return target;
12 }
```

常见坑：

- 使用 `JSON.parse(JSON.stringify())` 无法处理函数、正则和循环引用
- 使用 `Map` 而不是 `WeakMap` 导致内存无法及时回收

5. 浏览器中的事件循环（Event Loop）是如何工作的？

难度：★★

考点：JS 执行机制

答案要点：

事件循环是 JS 处理异步任务的核心机制，通过任务队列（Task Queue）和微任务队列（Microtask Queue）协调代码执行。

- 执行栈清空后，优先执行所有微任务（`Promise.then`, `MutationObserver`）
- 微任务清空后，取出宏任务队列中的第一个任务执行（`setTimeout`, I/O）
- 每一轮宏任务执行完后，会检查并执行产生的微任务
- 浏览器会在合适的时机进行 UI 渲染

常见坑：

- 混淆 Node.js 和浏览器的事件循环差异
- 认为 `Promise` 的构造函数内部代码是异步的

追问：

- `requestAnimationFrame` 属于宏任务还是微任务？

6. TypeScript 中的 interface 和 type 有什么区别？

难度：★

考点：TS 基础

答案要点：

两者在大多数场景下可以互换，但 `interface` 侧重于描述对象结构，`type` 侧重于定义类型别名。

- 扩展性：`interface` 可以通过重复定义自动合并（Declaration Merging），`type` 不行
- 组合方式：`interface` 使用 `extends`，`type` 使用交叉类型（`&`）
- 能力：`type` 可以定义基本类型别名、联合类型、元组，`interface` 只能定义对象
- 报错信息：`interface` 的错误提示通常更友好

常见坑：

- 在需要利用声明合并扩展第三方库类型时使用了 `type`

追问：

- 什么是 TS 的泛型约束（Generic Constraints）？

二面（深入原理与项目）

1. AI 聊天界面的流式文本渲染，如果内容非常长，如何保证页面不卡顿？

难度：★★★

考点：性能优化、DOM 渲染

答案要点：

长文本流式渲染的瓶颈在于频繁的 DOM 更新和长列表导致的重排重绘，需要从渲染频率和 DOM 节点数入手。

- 节流渲染：不要每收到一个字符就更新 DOM，可以使用 requestAnimationFrame 批量更新
 - 虚拟滚动：当历史对话过多时，只渲染可视区域内的消息卡片
 - 增量更新：仅对当前正在生成的文本节点进行 append，避免重绘整个容器
 - 文本分片：对 Markdown 解析进行增量处理，避免重复解析已生成的内容
- 常见坑：

- 每次收到新字符都调用 setState 导致整个 React 组件树重绘
 - Markdown 渲染库在处理超长字符串时性能指数级下降
- 追问：
- 如何在流式输出时实现自动滚动到底部，且不干扰用户手动向上滚动？

2. 谈谈 React 的 Fiber 架构，它解决了什么问题？

难度：★★★

考点：框架原理

答案要点：

Fiber 是 React 16 引入的新的协调引擎，将同步不可中断的更新变成了异步可中断的更新。

- 解决痛点：JS 引擎长时间占用主线程导致页面丢帧、卡顿
 - 核心机制：将更新任务拆分为多个小工单（Fiber 节点），在浏览器空闲时间执行
 - 优先级调度：通过 Scheduler 为不同任务分配优先级（如用户输入高于数据请求）
 - 双缓存技术：在内存中构建 workInProgress 树，完成后一次性提交到真实 DOM
- 常见坑：

- 误认为 Fiber 提高了代码执行速度（实际上增加了工作量，但提高了响应性）
- 追问：
- React 的合成事件机制为什么要设计在 Fiber 架构中？

3. 在 AI 应用中，前端如何计算 Token 数量以防止超过模型上下文限制？

难度：★★

考点：AI 业务场景、算法

答案要点：

Token 并不是简单的字符计数，前端通常需要引入专门的编码库（如 gpt-tokenizer 或 tiktoken）进行估算。

- 编码原理：基于 BPE（Byte Pair Encoding）算法将文本切分为 token ID
- 库的选择：在浏览器端使用 WebAssembly 版本的 tiktoken 以保证性能
- 异步处理：由于编码计算量较大，建议放在 Web Worker 中执行
- 预估策略：如果不需要精确，英文可按 1 token $\approx$ 4 字符，中文约 1-2 字符预估

常见坑：

- 直接用 string.length 作为 token 数传给后端
- 在主线程同步计算长文本的 token 导致 UI 冻结

4. 如何实现一个具有“打字机效果”的 AI 响应组件？

难度：★★

考点：组件设计、动画

答案要点：

打字机效果需要结合流式数据接收和前端的平滑展示逻辑。

- 缓冲区机制：建立一个队列存储后端传来的字符，前端按固定频率消费
- 动态速率：根据队列长度动态调整打字速度，防止数据堆积或停顿
- 样式处理：使用 CSS 伪元素实现闪烁的光标
- 停止逻辑：支持手动中断流传输并停止动画

常见坑：

- 渲染速度赶不上接收速度，导致页面在对话结束后还在“打字”
- 忽略了 Markdown 标签（如代码块）被截断时的渲染异常

5. 你们项目中的前端工程化是怎么做的？特别是针对 AI 频繁变动的 Prompt。

难度：★★

考点：工程化、Prompt 管理

答案要点：

针对 AI 业务，工程化不仅包含构建和部署，还包括对模型参数和 Prompt 的版本管理。

- Prompt 抽离：将 Prompt 从业务代码中抽离，作为配置文件或通过 CMS 管理
- 自动化测试：引入 LLM 评测链路，对前端 Prompt 改动进行回归测试
- 环境隔离：针对不同模型版本（GPT-4 vs GPT-3.5）配置不同的前端逻辑开关
- 监控告警：监控 AI 接口的首字响应时间（TTFT）和异常断开率

常见坑：

- 将复杂的 Prompt 硬编码在 onClick 回调函数里
- 缺乏对 AI 响应格式（如 JSON）的强校验

6. 什么是 RAG（检索增强生成）？前端在 RAG 流程中可以做哪些优化？

难度：★★★

考点：AI 架构、用户体验

答案要点：

RAG 通过检索外部知识库来增强 LLM 的回答准确性，前端主要负责引用来源的展示和交互。

- 引用标注：在文本中高亮显示知识来源，并支持点击跳转到原文位置
- 预检索优化：在用户输入时进行关键词联想，提前触发向量搜索
- 结果反馈：提供赞/踩功能，收集用户对检索质量的反馈用于微调
- 预览功能：在侧边栏提供 PDF 或文档的实时预览，并定位到相关片段

常见坑：

- 引用链接失效或指向错误的文档片段
- 检索结果过多时，前端展示过于拥挤

7. 手写代码：实现一个带并发限制的 Promise 调度器。

难度：★★★

考点：异步控制、算法

答案要点：

需要维护一个运行中队列和一个等待队列，当运行中数量小于限制时，从等待队列取任务执行。

代码示例：

代码块

```
1  class Scheduler {
2    constructor(limit) {
3      this.limit = limit;
4      this.running = 0;
5      this.queue = [];
6    }
7    add(fn) {
8      return new Promise((resolve) => {
9        this.queue.push({ fn, resolve });
10       this.run();
11     });
12   }
13   run() {
14     while (this.running < this.limit && this.queue.length) {
15       this.running++;
16       const { fn, resolve } = this.queue.shift();
```

```
17     fn().then(() => {
18         this.running--;
19         resolve();
20         this.run();
21     });
22 }
23 }
24 }
```

8. 谈谈 Web Worker 的使用场景，在 AI 前端应用中它能做什么？

难度：★★

考点：多线程、性能

答案要点：

Web Worker 为前端提供了多线程能力，可以将高耗时任务从主线程剥离。

- 场景 1：运行轻量级的本地模型（如 Transformers.js）进行文本分类或向量化
- 场景 2：处理大规模 Markdown 渲染或复杂的代码高亮计算
- 场景 3：进行长文本的 Tokenizer 编码计算
- 场景 4：对 AI 生成的图片数据进行像素级处理

常见坑：

- 在 Worker 中尝试操作 DOM
- 频繁的大数据量 PostMessage 导致的序列化开销

三面（架构与综合能力）

1. 如果让你从零设计一个企业级的 AI Chat 桌面端应用，你会如何进行技术选型？

难度：★★★

考点：系统架构、技术选型

答案要点：

选型需要平衡开发效率、性能和跨平台需求。

- 核心框架：Electron 或 Tauri（若追求安装包体积和性能选 Tauri）
- UI 方案：React + Tailwind CSS（快速构建响应式 AI 界面）
- 状态管理：Zustand（轻量且支持多窗口状态同步）
- 本地存储：SQLite 或 IndexedDB（存储海量历史对话，支持全文搜索）

- 通信机制：HTTP/2 SSE 处理流式输出，tRPC 保证前后端类型安全

常见坑：

- 忽略了桌面端应用在处理本地文件系统时的安全性
- 没考虑多窗口模式下的模型配置同步问题

2. 如何设计一个通用的 AI 组件库？需要考虑哪些特殊组件？

难度：★★★

考点：组件化、抽象能力

答案要点：

AI 组件库不同于传统 UI 库，它更强调流式交互和多模态输入。

- 核心组件：PromptInput（支持斜杠命令、文件上传）、MessageList（支持流式渲染、代码块、数学公式）、ThoughtChain（思维链折叠展示）
- 交互抽象：封装 useChat Hooks，统一处理流式解析、错误重试、中断请求
- 扩展性：支持自定义渲染器（Renderer），如针对特定领域的图表渲染
- 性能：内置虚拟列表和增量渲染逻辑

追问：

- 如何处理 AI 生成内容中的 XSS 攻击风险？

3. 在 AI 落地过程中，前端工程师的价值体现在哪里？

难度：★★

考点：职业思考、行业洞察

答案要点：

前端不仅是界面的实现者，更是 AI 能力与用户需求之间的桥梁。

- 交互创新：设计更符合直觉的 AI 交互方式（如 Copilot 模式、内联编辑）
- 性能把控：通过流式处理、本地缓存、预加载等手段抹平模型响应慢的痛点
- 边界处理：处理 AI 的幻觉输出，通过 UI 引导用户提供更准确的 Prompt
- 业务集成：将原子化的 AI 能力深度集成到现有的复杂业务工作流中

常见坑：

- 认为前端只是调个接口展示文本，忽略了 AI 带来的交互范式变革

4. 谈谈你对 Prompt Injection（提示词注入）的理解，前端能做哪些防护？

难度：★★★

考点：安全、AI 风险

答案要点：

提示词注入是指用户通过输入特殊指令误导模型忽略原指令，执行恶意操作。

- 输入过滤：在前端对用户输入进行敏感词扫描或正则匹配
- 结构化输入：尽量使用 ChatML 格式，明确区分 System、User 和 Assistant 角色
- 权限隔离：AI 生成的代码或脚本在沙箱环境（如 iframe 或 Web Worker）中运行
- 输出转义：对 AI 返回的内容进行严格的内容安全策略（CSP）校验

常见坑：

- 盲目信任 AI 返回的 HTML 片段并直接使用 dangerouslySetInnerHTML

5. 当 AI 接口响应非常慢（TTFT 超过 5 秒）时，你会通过哪些手段优化用户体验？

难度：★★

考点：用户体验、性能优化

答案要点：

在无法改变模型速度的情况下，通过感知优化降低用户的焦虑感。

- 状态反馈：提供多阶段的 Loading 提示（如“正在检索文档...”、“正在思考...”）
- 预加载：根据用户输入的前几个字，预判意图并提前建立连接
- 乐观更新：先展示用户的提问，并立即显示占位符
- 离线处理：对于常见问题，先通过本地轻量级模型或缓存给出即时响应

追问：

- 如果后端支持，如何利用 HTTP 预连接（Preconnect）优化首包时间？

6. 聊聊你在过去一年中遇到的最难的 AI 前端技术问题，你是如何解决的？

难度：★★★

考点：问题解决能力、项目深度

答案要点：

（此题无固定答案，需结合实际经验）

- 描述背景：例如在处理超长代码 Diff 时的性能问题，或多模态数据的同步展示
- 解决过程：分析瓶颈（如 DOM 节点过多、正则回溯）、对比方案（Canvas 渲染 vs 虚拟列表）
- 最终结果：量化的性能提升（如 FPS 从 20 提升到 60）或用户满意度提升
- 经验总结：沉淀出的通用组件或技术方案

7. 你如何看待 WebAssembly 在未来 AI 前端领域的发展？

难度：★★★

考点：技术视野

答案要点：

Wasm 是前端处理计算密集型 AI 任务的关键技术。

- 本地推理：通过 Wasm 运行 ONNX 或 TensorFlow.js 模型，实现隐私保护和离线可用
- 性能跨越：在 Tokenizer、向量计算、图像预处理等场景提供接近原生的速度
- 跨语言复用：将 Python 或 C++ 编写的成熟 AI 工具库直接移植到浏览器
- 趋势：随着 WebGPU 的普及，Wasm + WebGPU 将使浏览器具备处理中型模型的能力

8. 假如项目需要支持“多轮对话的上下文管理”，前端在存储和传输上有什么策略？

难度：★★

考点：状态管理、数据结构

答案要点：

上下文管理需要平衡模型的 Token 限制和对话的连贯性。

- 存储策略：使用 IndexedDB 存储完整的历史记录，内存中仅保留当前活跃的上下文
- 传输策略：前端负责根据 Token 限制裁剪历史记录（如滑动窗口法、摘要压缩法）
- 状态同步：确保多设备或多页面间的对话状态一致性
- 引用管理：为每条消息分配唯一 ID，支持针对特定历史消息的追问或修改

常见坑：

- 每次都发送全部历史记录，导致 Token 消耗过快且触发长度限制
- 忽略了用户删除中间某条消息后对后续上下文的影响